情報科学演習 D

課題 2 レポート

【担当教員】　桝井 晃基，Olivier Nourry　教員

【提出者】　向井 大貴　(09B23081)
ソフトウェア科学コース・3 年
u230984j@ecs.osaka-u.ac.jp

【提出日】　2025 年 11 月 14 日

1　システムの仕様

`Parser.run(String)` に字句解析器を実装する．第一引数に与えられた ts ファイルを読み込み, 構文的な誤りがあるかを判定する．構文に問題がなかった場合は'OK' という文字列を返し, 構文が正しくない場合は，"Syntax error: line" という文字列とともに，最初の誤りを見つけた行の番号を返す. なお, 入力ファイルが見つからなかった場合は,"File not found" という文字列を返す．独自仕様として以下を定義する．

2　課題達成の方針と設計

2.1　方針

パーサでは, 字句解析の結果をもとに AST(抽象構文木) を構築する．もし構文に誤りがある場合,AST の構築に失敗するため, その時点でエラーを検出できる．AST は今後の課題で使用するため, 構文解析の段階で AST を構築することにした．

今回課題となっている pascal のサブセット言語は, 基本的に先読みすることなく構文解析できる. しかし, 最初のトークンが識別子である場合のみ, それが代入文の一部なのか, 手続き呼び出し文の一部なのかを判断するために 1 トークン先読みする．

2.2　設計

AST は, 構文を構成する各要素をノードとして表現する．ノードは主に, 式 (expr), 文 (stmt), その他の 3 種類に分類される．実装するノードは表 1 のとおりである. 表では, 式には [E], 文には [S] を付与して区別する．なお, ノードを分割する粒度は, 課題 4 で中間コード生成を行うことを考慮して調整している. 詳細は課題 4 のレポートで説明することとする．

表 1: AST のノード

ノード	子	説明
Program	Block,CompoundStmt	ルート
Block	{VarDecl},{SubProgDecl}	宣言部分
VarDecl	Type	変数宣言
ArrayType	Expr	配列型
StandardType		標準型
SubProgDecl	SubProgHeader,{VarDecl},CompoundStmt	副プログラム宣言
SubProgHeader	{FormalParam}	副プログラム宣言のヘッダー
FormalParam	Type	形式パラメータ
[S]Assignment	Var,Expr	代入文
[S]Call	{Expr}	手続き呼び出し文
[S]CompoundStmt	{Stmt}	複文
[S]If	Expr,CompoundStmt,CompoundStmt	条件分岐文
[S]Read	{Var}	入力文
[S]While	Expr,CompoundStmt	繰り返し文
[S]Write	{Expr}	出力文
[E]BinaryOp	Expr,Expr	二項演算式
[E]Const		定数
[E]IndexedVar	Expr	配列要素
[E]StandardVar		標準変数
[E]UnaryOp	Expr	単項演算式

3　実装

3.1　全体の概要

パーサの実装について, 処理の流れを簡単に説明する. まず, 入力ファイルを読み込み, `Token` クラスのリストとして字句解析結果を取得する. トークンファイルの読み込み及びクラスなどは前回の課題で実装済みであり, 詳細はレポート 1 を参照のこと.

次に, 取得したトークンリストをもとに構文解析を行う. 構文解析は再帰下降構文解析で実装しており, 各非終端記号に対応するメソッドを用意している. 例えば, 非終端記号 `program` に対応するメソッドは `parseProgram()` である. 各メソッドは, 対応する非終端記号の構文規則に従ってトークンを消費し, 必要に応じて他の非終端記号に対応するメソッドを呼び出す. 構文解析が成功した場合, 対応する AST ノードを生成し, 呼び出し元に返す. 構文解析中に構文エラーが発生した場合, 例外 `AstParseException` をスローし, エラーメッセージと行番号を取得して返す.

AST ノードの定義は, まず, 抽象クラス `AstNode` を定義し, 各ノードはこのクラスを継承する形で実装した. また, 式や文など, 共通の性質を持つノードについては, それぞれ AstNode を継承する抽象クラス `Expr` や `Stmt` を定義し, それらを継承する形で実装した. 例えば, 代入文ノードは `AssignmentNode` クラスとして実装し, `Stmt` を継承している.

3.2　実装の詳細

パーサの実装

パーサの実装は, 主に `AstParser` クラスで行っている. このクラスは, コンストラクタでトークンリストを受け取り, メソッド `parse()` で構文解析を開始する. 内部には, 現在のトークン位置を示すインデックスを持っており, トークンの取得や消費を行うためのヘルパーメソッドを実装している. ヘルパメソッドの一覧を表 2 に示す. ”プログラム ” のパースを行う `parseProgram()` メソッドの実装例を以下に示す.

```
// プログラム = "program" プログラム名 ";" ブロック 複合文 "." と定義される
private ProgramNode parseProgram() {
    Token start = expect(TokenDef.PROGRAM); // 'program'を期待
    String programName = expect(TokenDef.IDENT).token; // 識別子を期待
    expect(TokenDef.SEMICOLON); // ';'を期待
    BlockNode block = parseBlock(); // "ブロック"をパース
    CompoundStmtNode stmt = parseCompoundStmt(); // "複合文"をパース
    Token end = expect(TokenDef.DOT); // '.'を期待
    AstNode.Position position = AstNode.Position.fromToken(start, end);
    return new ProgramNode(position, programName, block, stmt); //ノードを返す
}
```

なお, 例の中にある `position` は,AST ノードの位置情報を表すために使用している. これにより, 意味チェッカーなど AST を利用する処理においてもエラーメッセージに行番号を含めることができる. 今回の課題ではノードが複数行にまたがることはないが, 将来的な拡張性のために, ノードの開始位置と終了位置を保持するようにしている.

表 2: パーサのヘルパーメソッド

メソッド	引数	説明
peek()	なし	現在のトークンを取得する
consume()	なし	現在のトークンを取得し, インデックスを進める (消費する)
check(TokenDef)	トークン	現在のトークンが引数と一致するかを判定する
expect(TokenDef)	トークン	引数と一致する場合にトークンを消費し, 一致しない場合に例外をスローする

式のパース

式のパースは少し複雑であるため, 詳細に説明する. 式に含まれうる要素としては, 定数, 変数, インデックス付き変数, 二項演算式, 単項演算式がある. まず, すべての式は二項演算式を最上位とし, `parseExpr()` メソッドでは, `parseBinaryOp()` メソッドを呼び出す.

二項演算式では, 演算子の優先順位に基づいて再帰的にパースを行う. ここで, 優先順位を正しく処理し二項演算を適切にネストさせなければ, 後続の意味解析やコード生成で問題が生じてしまう. 優先順位は,1:OR, 2:AND, 3:比較演算子, 4:+,-, 5:*,DIV, MOD となっており, 数字が大きいほど優先順位が高い. `parseBinaryOp` メソッドは, 引数として最小優先順位を受け取る. 最小優先順位以下の演算子が現れるまで, 最小優先順位を更新しつつ再帰的に二項演算のパースを行うことで, ネストを正しく表現した木を構築している. なお, 今回の構文規則には右結合の演算子は存在しないが, 将来的な拡張性を考慮して右結合の処理も実装している. 二項演算子のパースの実装を以下に示す.

```
    private Expr parseExpr() {
        return parseBinaryOp(1);
    }

    private Expr parseBinaryOp(int minPriority) {
        Token start = peek();
        Expr left = parseUnaryOp();
        while (true) {
            OperatorInfo opInfo
                = BINARY_OPERATORS.get(TokenDef.getTokenDef(peek().id));

            if (opInfo == null || opInfo.priority < minPriority) {
                break;
            }
            consume(); // operator

            int nextMinPriority
                = opInfo.rightAssociative ? opInfo.priority : opInfo.priority + 1;
            Expr right = parseBinaryOp(nextMinPriority);
            BinaryOpNode.Op op = opInfo.op;
            Token end = peek();
            AstNode.Position position = AstNode.Position.fromToken(start, end);
            left = new BinaryOpNode(position, op, left, right);
        }
        return left;
    }
```

次に, 単項演算子のパースについて説明する. 二項演算のパースで, 単項演算式をパースするために `parseUnaryOp()` メソッドが呼び出される. ここでは,+,-,NOT の 3 つの単項演算子を処理する. 単項演算子が現れた場合, トークンを消費し,UnaryOp ノードを生成する. 単項演算子が現れなかった場合, 式, 定数, 変数, インデックス付き変数のいずれかをパースするために `parsePrimary()` メソッドを呼び出す. `parsePrimary()` メソッドでは, 現在のトークンに応じて, 式, 定数, 変数, インデックス付き変数のいずれかをパースする. トークンと処理すべき内容の対応は表 3 のとおりである.

表 3: プライマリ式のトークン対応

トークン	対応する処理
”(”	式のパース
識別子	変数またはインデックス付き変数のパース
整数定数	定数のパース
”true”, ”false”	ブール定数のパース
文字列定数	文字列定数のパース
それ以外	構文エラー

式のパース

式のパースは, すでに述べたように先読みを行うため, 注意が必要である. 識別子が現れた場合, 次のトークンによって, それが代入文なのか手続き呼び出し文なのかが決まる. そのため, `parseStatement()` メソッドでは, 識別子が現れた場合に次のトークンを先読みし, それに応じて代入文か手続き呼び出し文のパースを行う.

```
    private Stmt parseStatement() {
        if (check(TokenDef.IDENT)) {
            Token nextToken = tokens.get(pos + 1); // 1 トークン先読み
            if (nextToken.id == TokenDef.ASSIGN.id ||
                                nextToken.id == TokenDef.LBRACKET.id) {
                return parseAssignment(); // 代入文のパース
            } else if (nextToken.id == TokenDef.LPAREN.id ||
                                nextToken.id == TokenDef.SEMICOLON.id) {
                return parseCall(); // 手続き呼び出し文のパース
            } else {
                throw new AstParseException("Invalid statement", nextToken);
            }
        } ...
    }
```

Visitor の実装

AST ノードの走査を容易にするため,Visitor パターンを導入した. 各ノードクラスは, 抽象クラス `AstNode` に定義された `accept(AstVisitor)` メソッドをオーバーライドしている. Visitor インターフェース `IAstVisitor` は, 各ノードクラスに対応する訪問メソッドを定義している. 例えば, 代入文ノード `AssignmentNode` に対応する訪問メソッドは `visit(AssignmentNode)` である. これにより,AST に対する処理を AST ノードクラスから分離し, 新しい処理を追加する際にも既存のコードに影響を与えないため, 拡張性と保守性が向上する. Visitor パターンを利用した AST ノードの走査は, 今後の課題のレポートで詳細に説明することとする.

4 考察・工夫点

課題一から引き続き, タスクランナーの導入やテストの自動化などを行っている. 詳細はレポート 1 を参照のこと.

今後を見据えた実装

課題 1 に引き続き, 今後の実装を考慮し, 拡張性と保守性を重視した設計を心がけた. 例えば,AST ノードの設計においては, 各ノードが共通の抽象クラスを継承する形で実装することで, 新しいノードを追加する際にも既存のコードに影響を与えにくくしている. また, パーサの実装においても, 各非終端記号に対応するメソッドを用意することで, 新しい文法ルールを追加しやすくしている.

AST は, 今後の課題で利用することを前提に設計を行っているため, 各ノードは今後必要になる情報を保持しており, ソースを s2 パッケージとは分離している. また,AST は意味解析やコード生成などで走査されることを考慮し, あらかじめ Visitor パターンを導入している.

5 感想

テストについて 実装当初は, 二項演算の演算子の優先順位を無視して AST を構築していたため, パーサのテストは通過するが, 意味解析のテストで落ちてしまった. 既存のテストケースでは不十分であるため, 追加のテストケースを追加すべきだった. また, 現状問題はないが, パースには若干の不安が残るため, すべてのノードとそのパースについて単体テストを用意すべきだと感じた.

Nullable について Java は null 安全ではない. 今回, 例えば If 文の else 節が省略可能であるため,else 節が存在しない場合に,If ノードの else 節フィールドに null を設定している. このような場合, 後続の処理で null 参照例外が発生する可能性があるため,Optional 型を利用して null 安全を確保する方法をとるべきだったと感じた. 今後修正したいと思う.